

⌘ Blockchain & Développement

M2 AL/IABD Année 2025-2026

TP 1

Environnement Hardhat & Foundry

Premier token ERC-20 — Tests unitaires — Gas profiling — Déploiement Sepolia

1. Contexte et objectifs pédagogiques

Ce premier TP pose les fondations outillage du module. Vous allez configurer un environnement de développement de niveau production, implémenter un token ERC-20 depuis zéro en appliquant dès le départ les meilleures pratiques de sécurité et d'optimisation gas, puis écrire des tests automatisés dans deux paradigmes différents (JavaScript/TypeScript avec Hardhat et Solidity natif avec Foundry).

TÂCHE — Compétences visées

- Configurer un workspace Hardhat + Foundry avec TypeScript, OpenZeppelin et un gas reporter
- Implémenter l'interface ERC-20 complète avec erreurs custom, unchecked arithmetic et pattern CEI
- Écrire des tests unitaires en TypeScript (Hardhat/Chai) couvrant les happy paths et les edge cases
- Écrire des tests en Solidity natif (Foundry) avec fuzzing et cheatcodes `vm.prank` / `vm.deal`
- Profiler le gas de chaque fonction et comparer des implémentations naive vs optimisée
- Déployer et vérifier un contrat sur le testnet Sepolia via Etherscan

2. Ressources et installation

2.1 Prérequis système

RESSOURCES — Logiciels à installer avant la séance

- Node.js v20+ LTS : <https://nodejs.org/en/download>
- Git : <https://git-scm.com/downloads>
- VS Code (recommandé) + extension Hardhat : <https://code.visualstudio.com>
- Extension Metamask dans Chrome/Firefox : <https://metamask.io>

2.2 Installation de Foundry

```
bash — Installation Foundry (Unix/macOS/WSL)
```

```
# Installation via foundryup (gestionnaire officiel)
curl -L https://foundry.paradigm.xyz | bash

# Relancer le terminal ou sourcer le profil, puis :
Foundryup

# Vérification des outils installés
forge --version    # forge 0.2.0 (xxxxxxx)
cast --version     # cast 0.2.0 (xxxxxxx)
anvil --version    # anvil 0.2.0 (xxxxxxx)

# Si vous êtes sur Windows sans WSL, utilisez :
# https://book.getfoundry.sh/getting-started/installation#on-windows
```

2.3 Faucets ETH Sepolia

Vous avez besoin d'ETH de test pour déployer sur Sepolia. Utilisez l'un de ces faucets gratuits :

- ❖ Alchemy Sepolia Faucet (recommandé) : <https://sepoliafaucet.com> — nécessite un compte Alchemy
- ❖ Infura Sepolia Faucet : <https://www.infura.io/faucet/sepolia>
- ❖ Google Cloud Web3 Faucet : <https://cloud.google.com/application/web3/faucet/ethereum/sepolia>

2.4 Clé API RPC (nœud Ethereum)

Vous avez besoin d'un nœud Ethereum pour déployer. Créez un compte gratuit sur Alchemy (<https://alchemy.com>) et créez une application Sepolia. Notez l'URL RPC de la forme :

```
https://eth-sepolia.g.alchemy.com/v2/VOTRE_CLE_API
```

3. Partie 1 — Initialisation du workspace

3.1 Création du projet Hardhat

Bash

```
# Créer le répertoire du projet
mkdir tp1-erc20 && cd tp1-erc20

# Initialiser npm et Hardhat en TypeScript
npm init -y
npm install --save-dev hardhat
npx hardhat init
# → Choisir : TypeScript project
# → Oui pour .gitignore et installer les dépendances

# Installer les dépendances supplémentaires
npm install --save-dev @nomicfoundation/hardhat-toolbox
npm install --save-dev @openzeppelin/contracts
npm install --save-dev hardhat-gas-reporter
npm install --save-dev dotenv
```

3.2 Fichier de configuration hardhat.config.ts

Remplacez le contenu du fichier hardhat.config.ts par la configuration suivante :

TypeScript — hardhat.config.ts

```
import { HardhatUserConfig } from 'hardhat/config';
import '@nomicfoundation/hardhat-toolbox';
import 'hardhat-gas-reporter';
import * as dotenv from 'dotenv';
dotenv.config();

const config: HardhatUserConfig = {
  solidity: {
    version: '0.8.24',
    settings: {
      optimizer: { enabled: true, runs: 200 },
      viaIR: true,
    },
  },
  networks: {
    hardhat: { chainId: 31337 },
    sepolia: {
      url: process.env.RPC_URL_SEPOLIA ?? '',
      accounts: process.env.PRIVATE_KEY ? [process.env.PRIVATE_KEY] : [],
      chainId: 11155111,
    },
  },
  etherscan: {
    apiKey: { sepolia: process.env.ETHERSCAN_API_KEY ?? '' },
  },
  gasReporter: {
    enabled: true,
    currency: 'EUR',
    outputFile: 'gas-report.txt',
    noColors: true,
  },
};
```

```
    },
  };

export default config;
```

3.3 Fichier .env

⚠ ATTENTION — Sécurité — Ne jamais committer le fichier .env

Créez un fichier .env à la racine du projet avec ce contenu.
Vérifiez que .env figure dans le .gitignore (il y est par défaut avec Hardhat).
Utilisez un wallet dédié au développement, jamais votre wallet principal.

```
bash — .env (à remplir avec vos valeurs)
RPC_URL_SEPOLIA=https://eth-sepolia.g.alchemy.com/v2/VOTRE_CLE
PRIVATE_KEY=0xVOTRE_CLE_PRIVEE_WALLET_TEST
ETHERSCAN_API_KEY=VOTRE_CLE_ETHERSCAN
```

3.4 Initialisation de Foundry dans le même workspace

```
bash
# Initialiser Foundry dans le projet existant
forge init --no-git --no-commit .

# Supprimer les fichiers d'exemple Foundry (on garde notre structure)
rm src/Counter.sol test/Counter.t.sol script/Counter.s.sol

# Créer les remappings pour OpenZeppelin
echo 'remappings = [' >> foundry.toml
echo '   "@openzeppelin/=node_modules/@openzeppelin/", ' >> foundry.toml
echo ']' >> foundry.toml

# Vérification
forge build # doit compiler sans erreur
```

4. Partie 2 — Implémentation du token MyToken (ERC-20)

4.1 Interface IERC20

Créez le fichier contracts/IERC20.sol. C'est l'interface standard que votre implémentation devra respecter.

```
Solidity — contracts/IERC20.sol (fourni — ne pas modifier)
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

/// @title IERC20 — Interface standard EIP-20
interface IERC20 {
    function totalSupply() external view returns (uint256);
    function balanceOf(address account) external view returns (uint256);
    function transfer(address to, uint256 amount) external returns (bool);
    function allowance(address owner, address spender) external view returns (uint256);
    function approve(address spender, uint256 amount) external returns (bool);
    function transferFrom(address from, address to, uint256 amount) external returns (bool);

    event Transfer(address indexed from, address indexed to, uint256 value);
    event Approval(address indexed owner, address indexed spender, uint256 value);
}
```

4.2 Squelette de MyToken à compléter

TÂCHE — Votre travail — Compléter les TODOs

Le fichier ci-dessous contient des blocs TODO à implémenter.
 Les fonctions `_transfer`, `_mint` et `_burn` sont déjà fournies — lisez-les attentivement.
 Respectez le pattern CEI (Checks-Effects-Interactions) partout.
 Utilisez `unchecked` là où c'est mathématiquement justifié.

Solidity — contracts/MyToken.sol (à compléter)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import './IERC20.sol';

/// @title MyToken — Token ERC-20 pédagogique
/// @notice Implémentation complète à réaliser en TP1
contract MyToken is IERC20 {

    // — Storage —————
    string public name;
    string public symbol;
    uint8 public immutable decimals;
    uint256 public totalSupply;
    address public owner;

    mapping(address => uint256) public balanceOf;
    mapping(address => mapping(address => uint256)) public allowance;

    // — Erreurs custom —————
    // TODO 1 : Déclarer les erreurs custom suivantes :
    // - Unauthorized() : appelant non autorisé
    // - InsufficientBalance(uint256 available, uint256 required)
    // - InsufficientAllowance(uint256 available, uint256 required)
    // - ZeroAddress() : transfert vers address(0)
    // - ZeroAmount() : montant nul

    // — Events supplémentaires —————
    event OwnershipTransferred(address indexed previousOwner, address indexed newOwner);

    // — Modificateurs —————
    // TODO 2 : Déclarer le modifier onlyOwner
    // qui revert avec Unauthorized() si msg.sender != owner

    // — Constructor —————
    constructor(
        string memory _name,
        string memory _symbol,
        uint8 _decimals,
        uint256 _initialSupply
    ) {
        // TODO 3 : Initialiser name, symbol, decimals, owner (= msg.sender)
        // puis appeler _mint(msg.sender, _initialSupply)
    }

    // — Fonctions ERC-20 publiques —————
    function transfer(address to, uint256 amount) external returns (bool) {
        // TODO 4 : Vérifier to != address(0) (revert ZeroAddress)
        // puis appeler _transfer(msg.sender, to, amount)
        // retourner true
    }

    function approve(address spender, uint256 amount) external returns (bool) {
        // TODO 5 : Vérifier spender != address(0)
        // Mettre à jour allowance[msg.sender][spender]
        // Émettre Approval(msg.sender, spender, amount)
        // retourner true
    }
}
```

```

    }

    function transferFrom(
        address from,
        address to,
        uint256 amount
    ) external returns (bool) {
        // TODO 6 : Implémenter transferFrom
        // 1. Vérifier to != address(0)
        // 2. Lire l'allowance : uint256 allowed = allowance[from][msg.sender]
        // 3. Si allowed != type(uint256).max (infinite approval) :
        //     vérifier allowed >= amount (revert InsufficientAllowance)
        //     puis décrémenter allowance[from][msg.sender] (avec unchecked)
        // 4. Appeler _transfer(from, to, amount)
        // 5. retourner true
    }

    // — Fonctions admin —————
    function mint(address to, uint256 amount) external onlyOwner {
        // TODO 7 : Appeler _mint(to, amount)
    }

    function burn(uint256 amount) external {
        // TODO 8 : Appeler _burn(msg.sender, amount)
    }

    function transferOwnership(address newOwner) external onlyOwner {
        // TODO 9 : Vérifier newOwner != address(0)
        // Émettre OwnershipTransferred(owner, newOwner)
        // Mettre à jour owner
    }

    // — Fonctions internes (FOURNIES — ne pas modifier) —————
    function _transfer(address from, address to, uint256 amount) internal {
        uint256 bal = balanceOf[from];
        if (bal < amount) revert InsufficientBalance(bal, amount);
        unchecked {
            balanceOf[from] = bal - amount;
            balanceOf[to] += amount;
        }
        emit Transfer(from, to, amount);
    }

    function _mint(address to, uint256 amount) internal {
        if (to == address(0)) revert ZeroAddress();
        if (amount == 0) revert ZeroAmount();
        totalSupply += amount;
        balanceOf[to] += amount;
        emit Transfer(address(0), to, amount);
    }

    function _burn(address from, uint256 amount) internal {
        uint256 bal = balanceOf[from];
        if (bal < amount) revert InsufficientBalance(bal, amount);
        unchecked {
            balanceOf[from] = bal - amount;
            totalSupply -= amount;
        }
        emit Transfer(from, address(0), amount);
    }
}

```

5. Partie 3 — Tests unitaires avec Hardhat (TypeScript)

5.1 Fichier de tests à compléter

Créez le fichier test/MyToken.test.ts. Complétez les sections marquées TODO.

TypeScript — test/MyToken.test.ts

```
import { expect } from 'chai';
import { ethers } from 'hardhat';
import { MyToken } from '../typechain-types';
import { SignerWithAddress } from '@nomicfoundation/hardhat-ethers/signers';

describe('MyToken', function () {
  let token: MyToken;
  let owner: SignerWithAddress;
  let alice: SignerWithAddress;
  let bob: SignerWithAddress;

  const NAME      = 'MyToken';
  const SYMBOL    = 'MTK';
  const SUPPLY    = ethers.parseEther('1000000'); // 1 million tokens

  beforeEach(async function () {
    [owner, alice, bob] = await ethers.getSigners();
    const Token = await ethers.getContractFactory('MyToken');
    token = await Token.deploy(NAME, SYMBOL, 18, SUPPLY);
  });

  // --- Déploiement ---
  describe('Deployment', function () {
    it('should set name, symbol and decimals correctly', async function () {
      // TODO A : Vérifier name, symbol et decimals
    });

    it('should mint initial supply to owner', async function () {
      // TODO B : Vérifier totalSupply == SUPPLY et balanceOf(owner) == SUPPLY
    });
  });

  // --- Transfer ---
  describe('transfer()', function () {
    it('should transfer tokens and emit Transfer event', async function () {
      const amount = ethers.parseEther('100');
      // TODO C : Vérifier l'événement Transfer + les balances après
    });

    it('should revert with InsufficientBalance if sender has not enough', async function () {
      // TODO D : alice essaie de transférer mais n'a rien
      // Utiliser revertedWithCustomError
    });

    it('should revert with ZeroAddress if recipient is address(0)', async function () {
      // TODO E
    });
  });

  // --- Approve / TransferFrom ---
  describe('approve() and transferFrom()', function () {
    it('should allow transferFrom after approve', async function () {
      // TODO F : owner approve alice pour 500 tokens
      //           alice transferFrom owner vers bob
      //           Vérifier les balances et l'allowance restante
    });

    it('should revert if allowance exceeded', async function () {
      // TODO G
    });
  });
});
```

```

    });

    it('should support infinite approval (uint256.max)', async function () {
        // TODO H : approve avec ethers.MaxUint256, vérifier que l'allowance
        //             ne diminue pas après transferFrom
    });
});

// --- Mint / Burn ---
describe('mint() and burn()', function () {
    it('owner can mint additional tokens', async function () {
        // TODO I
    });

    it('non-owner cannot mint', async function () {
        // TODO J : alice essaie de minter — doit revert Unauthorized()
    });

    it('any user can burn their own tokens', async function () {
        // TODO K
    });
});
});
});

```

5.2 Exécution des tests

```

bash
# Lancer tous les tests avec rapport gas
npx hardhat test

# Avec couverture de code
npx hardhat coverage

# Générer les types TypeScript depuis les ABIs
npx hardhat typechain

```

6. Partie 4 — Tests Foundry (Solidity + Fuzzing)

Créez le fichier `test/MyToken.t.sol`. Foundry reconnaît les fichiers de test par l'extension `.t.sol`.

```

Solidity — test/MyToken.t.sol (à compléter)
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import 'forge-std/Test.sol';
import '../contracts/MyToken.sol';

contract MyTokenTest is Test {
    MyToken token;
    address owner = makeAddr('owner');
    address alice = makeAddr('alice');
    address bob = makeAddr('bob');
    uint256 SUPPLY = 1_000_000 ether;

    function setUp() public {
        vm.prank(owner);
        token = new MyToken('MyToken', 'MTK', 18, SUPPLY);
    }

    // --- Tests unitaires ---
    function test_InitialState() public view {
        // TODO 1 : Vérifier name, symbol, decimals, totalSupply, balanceOf[owner]
    }
}

```

```

function test_Transfer() public {
    uint256 amount = 100 ether;
    vm.prank(owner);
    // TODO 2 : Transférer et vérifier les balances
}

function test_RevertIf_InsufficientBalance() public {
    // TODO 3 : Utiliser vm.expectRevert avec l'erreur custom encodée
    // vm.expectRevert(abi.encodeWithSelector(MyToken.InsufficientBalance.selector, 0, 1
ether))
}

// --- Tests de fuzzing ---
/// @notice Le fuzzing génère 256 valeurs aléatoires pour (to, amount)
function testFuzz_TransferConservesTotalSupply(
    address to,
    uint256 amount
) public {
    // TODO 4 : Filtrer les cas invalides avec vm.assume
    // vm.assume(to != address(0) && to != owner)
    // Borner amount : amount = bound(amount, 0, SUPPLY)
    // Vérifier que totalSupply reste identique après le transfert
}

function testFuzz_ApproveAndTransferFrom(
    uint256 approveAmt,
    uint256 transferAmt
) public {
    // TODO 5 : Fuzzing sur approve + transferFrom
    // Si transferAmt > approveAmt → doit revert
    // Si transferAmt <= approveAmt → doit réussir
}

// --- Invariants ---
/// @notice Foundry appellera cet invariant après chaque séquence d'actions
function invariant_totalSupplyIsConsistent() public view {
    // TODO 6 : Vérifier que totalSupply == balanceOf[owner] + balanceOf[alice] + ...
    // Pour un invariant simple : assertEq(token.totalSupply(), SUPPLY); // pas de
mint/burn dans ce test
    // (Pour un invariant complet il faudrait un Handler, voir la documentation Foundry)
}
}

```

6.2 Commandes Foundry

```

bash
# Compiler les contrats
forge build

# Lancer tous les tests
forge test -vv

# Lancer avec traces détaillées
forge test -vvvv
# Fuzzing avec 1000 runs (défaut = 256)
forge test --fuzz-runs 1000

# Rapport de gas
forge test --gas-report
# Coverage
forge coverage --report summary

# Snapshot gas (à inclure dans le rapport)
forge snapshot

```

7. Partie 5 — Déploiement et vérification sur Sepolia

7.1 Script de déploiement Hardhat

Créez le fichier scripts/deploy.ts :

TypeScript — scripts/deploy.ts

```
import { ethers, run, network } from 'hardhat';

async function main() {
  const [deployer] = await ethers.getSigners();
  const balance = await deployer.provider.getBalance(deployer.address);

  console.log('Deploying from:', deployer.address);
  console.log('Balance      :', ethers.formatEther(balance), 'ETH');

  if (balance < ethers.parseEther('0.01')) {
    throw new Error('Balance insuffisante – récupérez des ETH Sepolia sur un faucet');
  }

  const SUPPLY = ethers.parseEther('1000000');
  const Token  = await ethers.getContractFactory('MyToken');
  console.log('Deploying MyToken...');
  const token  = await Token.deploy('MyToken', 'MTK', 18, SUPPLY);
  await token.waitForDeployment();

  const addr = await token.getAddress();
  console.log('MyToken deployed to:', addr);
  console.log('TX hash      :', token.deploymentTransaction()?.hash);

  // Vérification Etherscan (uniquement sur testnet/mainnet)
  if (network.name !== 'hardhat' && network.name !== 'localhost') {
    console.log('Waiting 5 blocks for Etherscan indexing...');
    await token.deploymentTransaction()?.wait(5);

    await run('verify:verify', {
      address: addr,
      constructorArguments: ['MyToken', 'MTK', 18, SUPPLY],
    });
    console.log('Contract verified on Etherscan!');
  }
}

main().catch(err => { console.error(err); process.exitCode = 1; });
```

7.2 Déploiement

bash

```
# Déploiement sur le réseau local Hardhat (test rapide)
npx hardhat run scripts/deploy.ts --network hardhat

# Déploiement sur Sepolia (nécessite .env configuré)
npx hardhat run scripts/deploy.ts --network sepolia

# Le script affiche l'adresse du contrat → copiez-la pour le rapport
```

8. Partie 6 — Analyse et optimisation gas (question de réflexion)

Exécutez `npx hardhat test` (qui génère `gas-report.txt`) et `forge snapshot` (qui génère `.gas-snapshot`). Analysez les résultats.

TÂCHE — Questions d'analyse à répondre dans le rapport

- Q1. Quel est le gas utilisé par la fonction `transfer()` dans votre implémentation ?
- Q2. Comparez `transfer()` avec et sans le bloc `unchecked` (décommentez-le temporairement). Quelle est l'économie en gas ?
- Q3. Comparez l'usage de `revert InsufficientBalance(...)` vs `require(condition, 'string')`. Testez les deux et mesurez.
- Q4. Quel est le gas du constructeur (déploiement) et quel en est le composant le plus coûteux ?
- Q5. Que se passe-t-il avec le gas si vous réactivez l'optimiseur avec `runs:1` vs `runs:200` vs `runs:1000` ?

9. Livrables attendus

LIVRABLE ATTENDU — Ce que vous devez remettre

- ① Repository GitHub public (ou privé avec accès partagé au professeur) :
 - › Fichier `README.md` avec : description du projet, instructions d'installation, commandes de test
 - › `contracts/MyToken.sol` — implémentation complète
 - › `test/MyToken.test.ts` — tests TypeScript (au moins 11 tests passants)
 - › `test/MyToken.t.sol` — tests Foundry (avec au moins 2 tests fuzz)
 - › `scripts/deploy.ts` — script de déploiement
 - › `.gas-snapshot` — généré par `forge snapshot`
 - › `gas-report.txt` — généré par `npx hardhat test`
- ② Rapport PDF (2 à 5 pages) contenant :
 - › Lien GitHub du repository
 - › Lien Etherscan Sepolia du contrat déployé et vérifié
 - › Réponses aux 5 questions d'analyse gas (Partie 6)
 - › Capture d'écran de `forge coverage` montrant la couverture de code
 - › Difficultés rencontrées et solutions trouvées

10. Grille d'évaluation

Critère	Barème	Détail
Implémentation <code>MyToken.sol</code>	/20	Tous les TODOs correctement implémentés, erreurs custom, <code>unchecked</code> justifié, <code>NatSpec</code>
Tests Hardhat TypeScript	/15	≥ 11 tests couvrant happy paths + edge cases + événements + custom errors
Tests Foundry + fuzzing	/15	≥ 2 tests fuzz avec <code>vm.assume</code> et <code>bound</code> correctement utilisés
Analyse gas (questions)	/20	5 réponses précises avec chiffres mesurés et comparaisons
Déploiement Sepolia vérifié	/15	Contrat visible et vérifié sur Etherscan Sepolia
Qualité du code et README	/10	Code lisible, commenté, README complet avec instructions
Bonus : <code>forge coverage</code> > 90%	+5	<code>Coverage branch</code> > 90% prouvé par <code>forge coverage --report summary</code>

11. Conseils et ressources complémentaires

✓ BONNE PRATIQUE — Ressources officielles

Hardhat documentation : <https://hardhat.org/docs>
Foundry Book (référence complète) : <https://book.getfoundry.sh>
OpenZeppelin Contracts : <https://docs.openzeppelin.com/contracts/5.x>
EIP-20 (standard ERC-20) : <https://eips.ethereum.org/EIPS/eip-20>
evm.codes (référence opcodes + gas) : <https://www.evm.codes>
Etherscan Sepolia : <https://sepolia.etherscan.io>

⚠ ATTENTION — Pièges fréquents

N'oubliez pas d'appeler `npx hardhat typechain` après chaque compilation pour régénérer les types TypeScript.
Le fichier `.env` ne doit JAMAIS être commité — vérifiez `.gitignore` avant `git push`.
Sur Sepolia, attendez 5 blocs de confirmation avant d'essayer de vérifier le contrat.
Le fuzz test avec `vm.assume` filtre les entrées invalides. Sans `vm.assume`, Foundry peut générer `address(0)` comme paramètre.
forge snapshot compare les résultats GAS entre les runs — si une PR augmente le gas, c'est visible immédiatement.